```
from collections import OrderedDict

class Node:
    def __init__(self, name, count, parent):
        self.name = name
        self.count = count
        self.parent = parent
        self.children = {}

def build_fp_tree(transactions, min_support):
    # Step 1: count support of each item
    item_count = {}
    for t in transactions:
        for item in t:
            item_count[item] = item_count.get(item, 0) + 1

    # Step 2: remove items below min_support
    item_count = {k: v for k, v in item_count.items() if v >= min_support}

    # Step 3: sort items by support (descending)
    order = sorted(item_count, key=lambda x: (-item_count[x], x))

    # Step 4: build the tree
    root = Node("Root", 0, None)
    for t in transactions:
        items = [i for i in t if i in item_count]
        items.sort(key=lambda x: order.index(x))

        node = root
        for item in items:
            if item in node.children:
                node.children[item].count += 1
            else:
                node.children[item] = Node(item, 1, node)
            node = node.children[item]

    return root, item_count, order

def print_tree(node, indent=0):
    if node.name != "Root":
        print("  " * indent + f"{node.name} ({node.count})")
    for child in node.children.values():
        print_tree(child, indent + 1)


# ---------- Scenario 1: Movie Streaming Platform ----------
transactions1 = [
    ["Action", "Comedy", "Thriller"],
    ["Action", "Drama", "Thriller"],
    ["Comedy", "Thriller", "Romance"],
    ["Action", "Comedy", "Thriller"],
    ["Action", "Comedy", "Drama"],
]

print("SCENARIO 1: Movie Streaming Platform")
root1, counts1, order1 = build_fp_tree(transactions1, min_support=3)
print("Frequent items:", counts1)
print("Order used:", order1)
print("FP-Tree:")
print_tree(root1)

print()

# ---------- Scenario 2: Smart Home IoT System ----------
transactions2 = [
    ["Smart Light", "Smart TV", "Air Conditioner"],
    ["Smart Light", "Smart Speaker", "Air Conditioner"],
    ["Smart TV", "Air Conditioner", "Smart Speaker"],
    ["Smart Light", "Smart TV", "Air Conditioner"],
    ["Smart Light", "Air Conditioner", "Smart Speaker"],
]

print("SCENARIO 2: Smart Home IoT System")
root2, counts2, order2 = build_fp_tree(transactions2, min_support=3)
print("Frequent items:", counts2)
print("Order used:", order2)
print("FP-Tree:")
print_tree(root2)
```

```
from collections import OrderedDict

class Node:
    def __init__(self, name, count, parent):
        self.name = name
        self.count = count
        self.parent = parent
        self.children = {}
```

```
def build_fp_tree(transactions, min_support):
    # Step 1: count support of each item
    item_count = {}
    for t in transactions:
        for item in t:
            item_count[item] = item_count.get(item, 0) + 1

    # Step 2: remove items below min_support
    item_count = {k: v for k, v in item_count.items() if v >= min_support}

    # Step 3: sort items by support (descending)
    order = sorted(item_count, key=lambda x: (-item_count[x], x))

    # Step 4: build the tree
    root = Node("Root", 0, None)
    for t in transactions:
        items = [i for i in t if i in item_count]
        items.sort(key=lambda x: order.index(x))

        node = root
        for item in items:
            if item in node.children:
                node.children[item].count += 1
            else:
                node.children[item] = Node(item, 1, node)
            node = node.children[item]

    return root, item_count, order
```

```
def print_tree(node, indent=0):
    if node.name != "Root":
        print("  " * indent + f"{node.name} ({node.count})")
    for child in node.children.values():
        print_tree(child, indent + 1)


# ---------- Scenario 1: Movie Streaming Platform ----------
transactions1 = [
    ["Action", "Comedy", "Thriller"],
    ["Action", "Drama", "Thriller"],
    ["Comedy", "Thriller", "Romance"],
    ["Action", "Comedy", "Thriller"],
    ["Action", "Comedy", "Drama"],
]

print("SCENARIO 1: Movie Streaming Platform")
root1, counts1, order1 = build_fp_tree(transactions1, min_support=3)
print("Frequent items:", counts1)
print("Order used:", order1)
print("FP-Tree:")
print_tree(root1)

print()

# ---------- Scenario 2: Smart Home IoT System ----------
transactions2 = [
    ["Smart Light", "Smart TV", "Air Conditioner"],
    ["Smart Light", "Smart Speaker", "Air Conditioner"],
    ["Smart TV", "Air Conditioner", "Smart Speaker"],
    ["Smart Light", "Smart TV", "Air Conditioner"],
    ["Smart Light", "Air Conditioner", "Smart Speaker"],
]

print("SCENARIO 2: Smart Home IoT System")
root2, counts2, order2 = build_fp_tree(transactions2, min_support=3)
print("Frequent items:", counts2)
print("Order used:", order2)
print("FP-Tree:")
print_tree(root2)
```

```
SCENARIO 1: Movie Streaming Platform
Frequent items: {'Action': 4, 'Comedy': 4, 'Thriller': 4}
```

```
Order used: ['Action', 'Comedy', 'Thriller']
FP-Tree:
  Action (4)
    Comedy (3)
      Thriller (2)
    Thriller (1)
  Comedy (1)
    Thriller (1)

SCENARIO 2: Smart Home IoT System
Frequent items: {'Smart Light': 4, 'Smart TV': 3, 'Air Conditioner': 5, 'Smart Speaker': 3}
Order used: ['Air Conditioner', 'Smart Light', 'Smart Speaker', 'Smart TV']
FP-Tree:
  Air Conditioner (5)
    Smart Light (4)
      Smart TV (2)
      Smart Speaker (2)
    Smart Speaker (1)
      Smart TV (1)
```